

Instagram Clone

Modules & Functional Requirements

Document Type Functional Specification	Stack Ruby on Rails
--	-------------------------------

This document outlines the modules and functional requirements for the Instagram Clone Rails test project. It is intended as a working specification for implementation: each module describes the expected behaviour, data ownership, and any constraints that affect database design or user-facing functionality. Implementation choices not explicitly mandated here are left to the developer's judgement, subject to standard Rails conventions and best practices.

1 Users

- A user can sign up and sign in.
- The **users** table must include all required information for authentication and profile display.
- A user can have a display picture (profile image).

2 Posts

- The **posts** table must include all required information (title/caption, author, timestamps, visibility, etc.).
- A user can create posts.
- A user can edit or delete their own posts.
- A post can have many images, up to a **maximum of 10**.

3 Stories

- A user can add stories.
- A user can delete their own stories.
- A story remains visible for **24 hours** from creation, unless deleted earlier by the user.
- Users can see each other's stories.

4 Comments & Likes

- The **comments** table must include all required information (author, post reference, body, timestamps).
- A post can have many comments.
- A user can comment on their own posts.
- A user can comment on other users' posts.
- A post's owner can edit or delete **any** comments on their own posts.
- A user can delete any of their own comments on any post.
- A user can like their own posts and other users' posts.

5 Followers

- Users can follow each other.
- A user account can be set to **private** or **public**.
- Private posts are only visible after the account owner accepts the follow request.

6 Search

- The application header includes a search field.
- As the user types, matching users are listed beneath the field.
- Clicking a result navigates to that user's show page.
- Search may be implemented with **Thinking Sphinx**, or any other full-text search technique. If **ILIKE / LIKE** is used, a proper **TRGM GIN index** must be implemented for efficient full-text matching.

Acceptance criteria. Each module above should be demonstrable end-to-end: a working UI flow, the corresponding database schema, and any required indexes or background jobs. Code is expected to follow standard Rails conventions, with strong parameters, scoped authorization, and N+1-safe queries in any feed or list view.